

Programiranje za web

Predavanje 6: Poslovna pravila i rokovi

doc. dr. sc. Tomislav Petković mag. ing. Josip Torić

12.11.2026.



Sveučilište u Zadru
Universitas Studiorum
Jadertina | 1396 | 2002 |

Plan predavanja

- ➊ Pravila koja shema ne može provjeriti
- ➋ Faze natjecanja: stanje izvedeno iz vremena
- ➌ Odjava i ponovna prijava
- ➍ Pogreška kao odgovor
- ➎ Testiranje pravila koja ovise o vremenu

Dosad su rute znale tko šalje zahtjev i smije li dirati taj zapis. Sada dolazi treće pitanje: smije li se radnja izvesti u ovom trenutku.

Ishodi učenja

Nakon ovog predavanja student može:

- razlikovati provjeru ulaznih podataka od poslovnog pravila i reći gdje koja stoji
- izvesti stanje zapisa iz vremena umjesto da ga pohranjuje
- napisati pravilo koje dopušta ili odbija radnju ovisno o fazi
- obrazložiti izbor između brisanja zapisa i promjene oznake stanja
- prevesti domensku pogrešku u odgovor s dosljednim kodom i porukom
- napisati test pravila koje ovisi o vremenu, bez čekanja

Domena primjera

Natjecanje ima dva roka: nominacijski i konačni. Oko njih se mijenja što klub smije:

Razdoblje	Što klub smije
prije prvog roka	prijaviti natjecatelja, mijenjati i odjaviti
između dvaju rokova	mijenjati i odjaviti postojeću prijavu, ali ne prijaviti novoga
nakon drugog roka	ništa od navedenoga

Popis natjecatelja koji organizator šalje sucima nastaje nakon konačnog roka. Sve što se tada promijeni, promijenilo bi i taj popis, pa izmjene prestaju.

Pravila koja shema ne može provjeriti

Razlika prema provjeri ulaznih podataka, mjesto pravila, odgovor koji ono vraća.

Dvije vrste provjere

Zahtjev za prijavom može biti odbijen iz dva različita razloga, a razlika nije u strogosti nego u tome što je trebalo pogledati:

	Provjera ulaza	Poslovno pravilo
Gleda	samo poslani sadržaj	stanje aplikacije
Mjesto	schemas	services
Primjer	kategorija izvan popisa	rok je prošao
Isti zahtjev sutra	jednako odbijen	može proći

Zadnji redak je i najkorisniji znak raspoznavanja. Neispravna kategorija neispravna je uvijek. Prijava nakon roka uredna je po obliku, a odbija se zbog trenutka u kojem je stigla.

Gdje pravilo stoji

Pravilo o roku ne može stajati u shemi, jer shema nema pristup bazi i ne zna koje je natjecanje u pitanju.

Ne može stajati ni u ruti. Rok vrijedi za prijavu, za izmjenu i za odjavu, dakle za tri rute, pa bi se ista provjera pisala tri puta i tri bi se puta mogla zaboraviti.

Ostaje sloj `services`: on ima sesiju baze, vidi cijelo stanje i nalazi se ispred svakog upisa. Zato sve tri rute pozivaju funkciju koja pravilo provodi na jednom mjestu.

Konvencija

Nijedan okvir ne propisuje gdje poslovna pravila stoje. Odluka da ih nosi sloj između rute i baze proizlazi iz podjele odgovornosti koju smo postavili na početku kolegija, a ne iz zahtjeva alata.

Primjer: kategorija i spol

Težinske kategorije IPF-a razlikuju se po spolu. Shema zna popis svih kategorija, pa nepostojeću odbija sama. Ne zna, međutim, kojem natjecatelju prijava pripada:

```
# app/services/registration_service.py
def _check_category(lifter, category):
    if category not in CATEGORIES_BY_GENDER[lifter.gender]:
        raise AppError(
            "invalid_category",
            f"Kategorija {category} ne odgovara spolu natjecatelja",
            400,
        )
```

Spol stoji u zapisu natjecatelja, a zahtjev nosi samo njegovu oznaku. Vrijednost se zato može prosuditi tek nakon dohvata iz baze, dakle ondje gdje baza postoji.

Faze natjecanja

Stanje izvedeno iz vremena, pomoćna funkcija, matrica dopuštenih radnji.

Stanje koje se ne pohranjuje

Natjecanje bi moglo imati stupac phase koji netko postavlja kad rok prođe. Taj bi stupac tražilo da ga netko mijenja: zadatak koji se pokreće u zadano vrijeme, ili ručni zahvat.

Dok se to ne dogodi, stupac laže. Rok je prošao u ponoć, a zapis još tvrdi da je natjecanje otvoreno, pa aplikacija prima prijave koje ne bi smjela.

Faza se zato ne pohranjuje nego **računa pri svakom zahtjevu**, usporedbom trenutnog vremena s dvama rokovima. Rokovi su podatak, faza je zaključak iz njih.

Cijena je jedna usporedba po zahtjevu, ali time osiguravamo da stanje nikada ne zaostane za vremenom.

Tri faze

```
# app/core/phases.py
class Phase(str, Enum):
    OPEN = "open"
    PRELIM_PASSED = "prelim_passed"
    CLOSED = "closed"
```

Faza**Vrijedi**

OPEN

prije nominacijskog roka

PRELIM_PASSED

između nominacijskog i konačnog roka

CLOSED

nakon konačnog roka

Faze su nabrojiv skup, pa ih Enum opisuje izravno. Usporedba tada glasi `phase == Phase.OPEN`, a ne usporedba dvaju datuma rasuta po kodu.

Funkcija koja fazu računa

```
# app/core/phases.py
def get_competition_phase(competition: Competition) -> Phase:
    now = datetime.now(timezone.utc)
    prelim = competition.prelim_deadline
    final = competition.final_deadline

    if now < prelim:
        return Phase.OPEN
    if now < final:
        return Phase.PRELIM_PASSED
    return Phase.CLOSED
```

Funkcija prima natjecanje i vraća fazu. Ne dira bazu, ne podiže iznimke i ne odlučuje što je dopušteno.

Usporedba se izvodi u zoni UTC. Rok bez zone uspoređivao bi se s vremenom druge zone, pa bi ishod ovisio o tome gdje poslužitelj radi.

Matrica dopuštenih radnji

Radnja	OPEN	PRELIM_PASSED	CLOSED
nova prijava	da	ne	ne
promjena kategorije	da	da	ne
odjava natjecatelja	da	da	ne
čitanje popisa	da	da	da

Faza u odgovoru

Matrica vrijedi na poslužitelju, ali i sučelje treba znati koje radnje ponuditi. Zato odgovor o natjecanju nosi i izračunatu fazu:

```
# app/schemas/competition.py
class CompetitionResponse(BaseModel):
    prelim_deadline: datetime
    final_deadline: datetime
    @computed_field
    @property
    def phase(self) -> Phase:
        return get_competition_phase(self)
```

Polje nije stupac u bazi nego vrijednost izvedena pri sastavljanju odgovora, pa ne može zaostati za vremenom.

Na zaštitu preko sučelja pritom se ne smijemo osloniti: zahtjev koji ipak stigne izvan faze odbija sloj services, bez obzira na to što je sučelje prikazalo.

Pravilo u servisu

```
# app/services/registration_service.py
comp = await _get_competition_or_404(db, comp_id)
phase = get_competition_phase(comp)
if phase != Phase.OPEN:
    raise AppError("deadline_passed",
                  "Rok za nove prijave je istekao", 400)
```

Provjera stoji prije svega ostaloga u funkciji, prije dohvata natjecatelja i prije provjere vlasništva. Zahtjev koji ne prolazi ne pokreće nijedan daljnji upit.

Uvjet je napisan kao `!= Phase.OPEN`, a ne kao nabrojanje faza u kojima se odbija. Doda li se jednom četvrta faza, ovo pravilo i dalje vrijedi bez izmjene.

Isto pravilo, drugi uvjet

Izmjena prijave dopuštena je i nakon nominacijskog roka, pa njezin uvjet imenuje samo fazu u kojoj se odbija:

```
# app/services/registration_service.py
comp = await _get_competition_or_404(db, comp_id)
phase = get_competition_phase(comp)
if phase == Phase.CLOSED:
    raise AppError("deadline_passed",
                  "Izmjene nisu čmogue nakon finalnog roka", 400)
```

Oba uvjeta koriste istu funkciju i isti kod pogreške, a razlikuju se u jednoj usporedbi. Time je matrica s prethodnog slajda doslovno prepisana u kod, redak po redak.

Odjava i ponovna prijava

Brisanje naspram promjene stanja, ponovljeni zahtjev, obrnuta radnja.

Odjava ne briše zapis

Klub koji odjavljuje natjecatelja mogao bi obrisati redak prijave. Nakon toga se više ne zna da je prijava postojala, ni kada je povučena.

To smeta na tri mjesta. Organizator ne može vidjeti povijest prijava. Klub koji se predomisli mora natjecatelja prijaviti iznova, a nova prijava nosi novo vrijeme upisa. Ograničenje jedinstvenosti više ne sprječava dvostruku prijavu, jer prvog retka nema.

Zato se zapis zadržava, a mijenja mu se oznaka stanja:

status	Značenje
active	prijava vrijedi i natjecatelj je na popisu
withdrawn	prijava je povučena, ali zapis o njoj ostaje

Odjava u kodu

```
# app/services/registration_service.py
reg = await registration_repo.get_by_id(db, reg_id)
if not reg or reg.competition_id != comp_id:
    raise AppError("not_found", "Prijava nije pronadena", 404)
if reg.status == "withdrawn":
    raise AppError("already_withdrawn", "Vec odjavljen", 400)

_check_lifter_ownership(reg, current_user)
...
reg.status = "withdrawn"
await db.flush()
```

Odjava nije DELETE nego POST na `/registrations/{id}/withdraw`. Adresa imenuje radnju jer riječ je o prijelazu stanja, a ne o uklanjanju resursa.

Isti postupak ima i obrnutu radnju: prijava se smije vratiti u `active` dok konačni rok nije prošao.

Dvostruki zahtjev

Korisnik koji ne vidi odgovor, jer je veza spora ili je stranica zapela, pritisnut će gumb još jednom. Drugi zahtjev stiže nad zapisom koji je već povučen.

Dva su moguća odgovora, i oba se susreću u praksi:

- **Tiho prihvatiti.** Stanje je već ono koje se tražilo, pa odgovor je 200 i ništa se ne mijenja. Radnja je time idempotentna.
- **Odbiti.** Drugi zahtjev dobiva 400 i kod `already_withdrawn`, jer traži prijelaz koji iz trenutnog stanja nije moguć.

Izbor, ne pravilo

U našem projektu drugi zahtjev se odbija, pa sučelje može reći korisniku da je odjava već provedena. Sučelje koje odabere tiho prihvaćanje lakše podnosi ponovljene zahtjeve, ali gubi razliku između prve i druge odjave.

Pogreška kao odgovor

Domenska pogreška, jedno mjesto prevođenja, izbor koda, popis kodova.

Domenska pogreška

Pravilo koje ne prolazi mora prekinuti obradu i doći do klijenta kao odgovor. Taj odgovor ne sastavlja sloj services: kad bi gradio HTTP odgovore, ne bi se mogao pozvati iz testa. Umjesto toga podiže iznimku koja nosi sve potrebno:

```
# app/core/errors.py
class AppError(Exception):
    def __init__(self, code: str, message: str,
                  status_code: int = 400):
        self.code = code
        self.message = message
        self.status_code = status_code
```

code je stalan i namijenjen kodu koji odlučuje što učiniti, a message čovjeku i smije se mijenjati bez posljedica.

Iznimka nosi i statusni kod, pa svako pravilo samo određuje je li njegov neuspjeh 400, 403, 404 ili 409.

Jedno mjesto prevođenja

Bez zajedničkog handlera svaka bi ruta morala hvatati iznimku i sama graditi odgovor. Umjesto toga, prijevod stoji na jednom mjestu:

```
# app/core/errors.py
async def app_error_handler(_request, exc: AppError) -> JSONResponse:
    return JSONResponse(
        status_code=exc.status_code,
        content={"code": exc.code, "message": exc.message},
    )

# app/main.py
app.add_exception_handler(AppError, app_error_handler)
```

Iznimka podignuta bilo gdje u obradi izlazi kroz ovaj handler, kao tijelo istoga oblika. Ruta o tome ne zna ništa.

Iznimka koja nije AppError ovdje se ne hvata nego daje 500. To je ispravno: nepredviđeno stanje ne smije izgledati kao uredno odbijen zahtjev.

Izbor koda u graničnim slučajevima

Kod 401 odnosi se na prijavu, a ne na prava: standard ga opisuje kao nedostatak valjanih podataka za prijavu, premda mu naziv glasi Unauthorized. Kod 403 znači da je pošiljatelj poznat, a zahtjev ipak odbijen. Ostatak izbora standard ne razrješava, jer njegovi opisi dopuštaju više od jednoga.

Konvencija, razlikuje se od projekta do projekta

- Neispravni podatci u tijelu zahtjeva: FastAPI i Pydantic vraćaju 422, ali je moguć i 400.
- Uspješno brisanje: 204 bez tijela ili 200 s podacima obrisano zapisa.
- Zapis koji postoji, ali ga korisnik ne smije vidjeti: 403 ili 404, ovisno o tome smije li se odati da zapis postoji.

Izbor se donosi jednom i dosljedno provodi kroz cijeli projekt.

Kodovi pogrešaka u projektu

Kod	HTTP	Kada
invalid_credentials	401	prijava ili token ne vrijede
token_expired	401	token je istekao
forbidden	403	uloga ili vlasništvo ne dopuštaju radnju
not_found	404	zapis ne postoji ili nije vidljiv
duplicate	409	zapis bi prekršio jedinstvenost
deadline_passed	400	radnja nije moguća u ovoj fazi
already_withdrawn	400	prijava je već povučena
already_active	400	prijava je već aktivna

Popis je kratak namjerno: klijent na svaki kod reagira drukčije.

Pogreška koju vraća baza

Ograničenje jedinstvenosti postavljeno uz model prijave ne javlja se kao uredan odgovor nego kao iznimka upravljačkog programa baze, pri izvođenju upisa:

```
# app/services/registration_service.py
try:
    await registration_repo.create(db, reg)
except IntegrityError:
    raise AppError("duplicate",
                   "Natjecatelj je vec prijavljen na ovo natjecanje", 409)
```

Bez ovog prevođenja klijent dobiva 500, jer je IntegrityError za aplikaciju nepredviđeno stanje. Taj je kod i netočan: zahtjev je u sukobu sa stanjem, a to je 409.

Provjera prije upisa ne bi bila dovoljna, jer se stanje može promijeniti između provjere i upisa. Ovdje odbija baza, a aplikacija njezinu pogrešku prepoznaje.

Testiranje pravila koja ovise o vremenu

Problem čekanja, zamrzavanje vremena, što se pokriva testom.

Pravilo koje se ne da isprobati

Pravilo o roku vrijedi samo u određenom razdoblju. Test koji ga želi provjeriti morao bi dočekati taj trenutak, a rokovi su udaljeni tjednima.

Dva su izlaza iz toga:

- **Pomaknuti podatke.** Natjecanju se u testu postave rokovi tik uz sadašnji trenutak. Test prolazi, ali ovisi o tome koliko dugo traje, pa postaje nepouzdan.
- **Pomaknuti vrijeme.** Trenutno vrijeme se za potrebe testa zamrzne na zadanu vrijednost, a podatci ostaju kakvi jesu.

Drugi je put pouzdaniji, jer test ne ovisi ni o trajanju ni o danu kad se pokreće.

Zamrznuto vrijeme

```
# api/tests/test_registrations.py
@freeze_time("2026-04-20 10:00:00")
async def test_create_registration_prelim_passed(
    client, club_and_user, lifter, competition,
):
    headers = await auth_header(client, "testclub", "klub123")
    resp = await client.post(
        f"/competitions/{competition.id}/registrations/",
        json={"lifter_id": lifter.id, "category": "83", "total": 500},
        headers=headers,
    )
    assert resp.status_code == 400
    assert resp.json()["code"] == "deadline_passed"
```

Natjecanje u testovima ima nominacijski rok 15.04.2026. i konačni 01.05.2026. Zamrznuto vrijeme 20.04. pada između njih, pa je faza PRELIM_PASSED i nova prijava mora biti odbijena.

Što test provjerava

Test ne gleda samo statusni kod nego i kod pogreške. Prva tvrdnja kaže da je zahtjev odbijen, druga da je odbijen iz očekivanog razloga.

Bez druge tvrdnje test bi prošao i da je zahtjev pao zbog neispravne kategorije ili nepostojećeg natjecatelja, dakle iz razloga koji s rokom nema veze.

Za svako pravilo potrebna su najmanje dva testa: jedan u kojem radnja prolazi i jedan u kojem je odbijena. Prvi pokazuje da pravilo ne blokira ono što bi trebalo dopustiti.

Odatle i broj testova za fazu: tri za novu prijavu, po jedan za svaku fazu, i dva za izmjenu, jer se ona razlikuje samo u zadnjoj.

Sažetak

- Provjera ulaza gleda poslani sadržaj, poslovno pravilo gleda stanje aplikacije; isti zahtjev sutra može proći.
- Pravilo stoji u sloju services, jer ima pristup bazi i nalazi se ispred svakog upisa, a vrijedi za više ruta.
- Faza se računa iz vremena pri svakom zahtjevu, pa ne može zaostati za stvarnim stanjem.
- Odjava mijenja oznaku stanja umjesto da briše zapis, čime povijest i ograničenje jedinstvenosti ostaju na snazi.
- Domenska pogreška nosi kod, poruku i statusni kod, a u odgovor je prevodi jedan handler.
- Ograničenje provodi baza, a aplikacija njezinu iznimku prevodi u 409.
- Pravilo koje ovisi o vremenu testira se zamrzavanjem vremena, uz tvrdnju o kodu pogreške, ne samo o statusu.

Što slijedi

- **19.11.2026.** Obrana prvoga dijela projekta.
- **26.11.2026.** Predavanje 7, arhitektura jednostranične aplikacije.
- **03.12.2026.** Predavanje 8, komunikacija sučelja s poslužiteljem.

Do obrane: prijaviti natjecatelja, odjaviti ga, pa isti zahtjev poslati još jednom i usporediti odgovore. Zatim postaviti nominacijski rok u prošlost i pokušati novu prijavu.